

PORTFOLIO // 2026 // ENGINEERING

DANIEL FAGBOHUNLU

Backend Engineer

I build backends from hardware protocols up to REST APIs, in Go, TypeScript, and Python.

HOW I WORK

01

AGILE STANDUPS

Daily standups to unblock dependencies and keep backend work aligned with team priorities.

02

GIT CONFLICTS

Resolve merge conflicts on shared API routes cleanly when building alongside other engineers.

03

LLM PAIRING

Use AI as a pair programmer to scaffold tests, validate Zod schemas, and speed up documentation.

SCHEMA CONSOLIDATION

PROBLEM

Separate Tenant and Agent models duplicated authentication logic and risked data-integrity issues.

APPROACH

Broke the schema changes down in Prisma, resolved Git conflicts with the team, and unified auth with JWT and Google OAuth2.

RESULT

Merged Tenant and Agent into a single User model with no data loss, across 30+ REST endpoints.

30+

REST ENDPOINTS

2 → 1 models unified

TypeScript · Express

PostgreSQL · Prisma

THREAD-SAFE AUTH

PROBLEM

Handling Keycloak tokens concurrently risked race conditions and added memory overhead under load.

APPROACH

Built a thread-safe AuthStore in Go with `sync.Map` and mutexes, and parsed dynamic Keycloak JSON with `tidwall/gjson` instead of reflection.

RESULT

A thread-safe auth layer across 4 flows: login, session validation, and OTP recovery, via token introspection.

4

AUTH FLOWS • KEYCLOAK

`sync.Map` + mutexes
`tidwall/gjson` parsing
token introspection

HARDWARE TELEMETRY

PROBLEM

The EMS needed live telemetry from inverters and battery systems speaking different serial protocols.

APPROACH

Read the Modbus, RS485, and CAN manuals, extracted the register addresses and data points, and wrote the polling and parsing in Go.

RESULT

A consistent Go telemetry path feeding live device metrics into the energy management system.

10+

INVERTER & BMS PRODUCTS

Modbus

RS485

CAN

TECHNICAL STACK

GO / EDGE

Concurrency sync.Map and mutexes for thread-safe pipelines.

Hardware Modbus, RS485, and CAN device telemetry.

JSON High-performance parsing with gjson, no reflection.

NODE / TYPESCRIPT

API Express and TypeScript with Zod validation.

Database PostgreSQL schema modeling with Prisma.

Testing Unit and end-to-end coverage with Jest.

LLM-ACCELERATED **WORKFLOW**

01

Generate Zod data-contracts mapped directly from Prisma models.

02

Scaffold Express route structures, then hand-refine the edge cases.

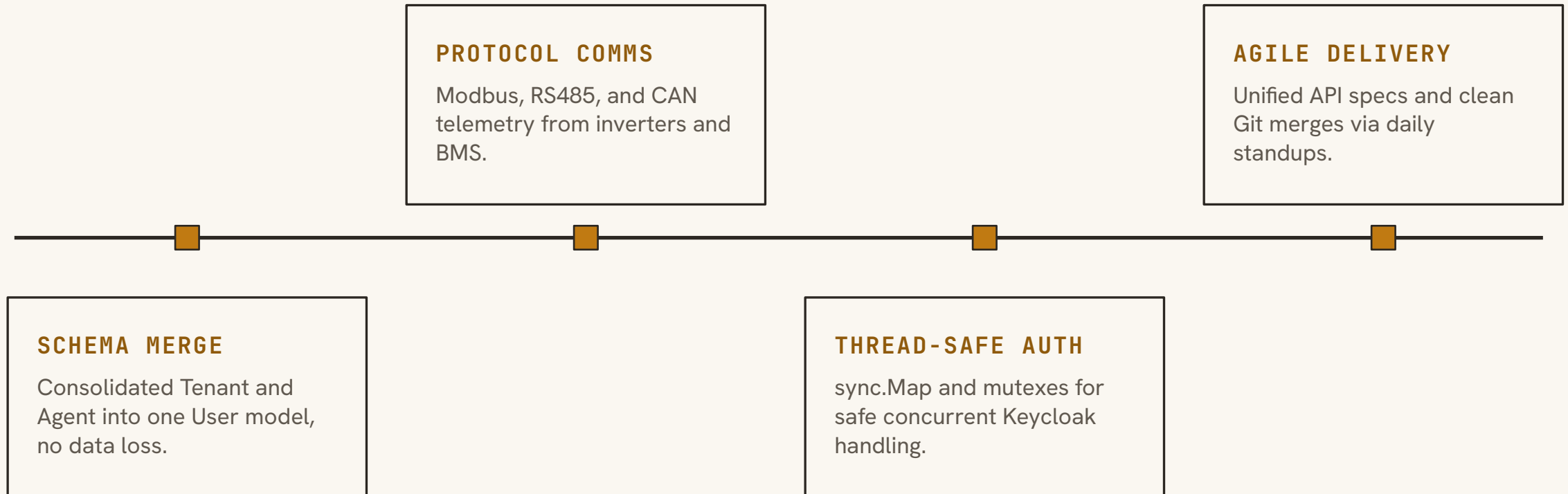
03

Decode cryptic compiler and database errors to isolate fixes fast.

04

Standardize documentation and generate regexes for custom sanitizers.

KEY MILESTONES



// CONTACT

INITIATE CONTACT

Open to systems and backend engineering roles. Always ready to build with a team.

dan.doyin@gmail.com



github.com/DantheCE



[linkedin.com/in/daniel-fagbohunlu](https://www.linkedin.com/in/daniel-fagbohunlu)

